

AEGIS.ai

Multi-Agent Application Security Analysis

Technical Documentation

Ritvik Indupuri

Abstract

AEGIS.ai is a React and Supabase application for AI-assisted application-security analysis. It combines interactive code, dependency, URL-string and prompt inspection; public GitHub repository scanning; vulnerability tracking; current CVE and Known Exploited Vulnerability context; and four streaming security-assistant roles behind a single interface.

The platform separates its browser management console from Supabase authentication, database and Edge Functions. Gemini 2.5 Flash performs structured scanner analysis and powers the fast agent roles, while OpenAI GPT-5 backs deeper code-review and threat-intelligence roles through the Lovable AI gateway. Authenticated records are isolated with row-level security and delivered to the dashboard through realtime subscriptions.

Document scope

- System architecture, agent routing and external-service boundaries.
- Implemented scan types, repository acquisition and finding enrichment.
- Agent modes, streamed responses, session history and usage controls.
- Data model, row-level security, scoring, scheduled-scan and alert paths.
- Build configuration, operational boundaries and source cross-reference.
- Prioritized next steps with concrete completion criteria.

Contents

Abstract	2
1 System overview	3
2 System architecture	4
3 Agent request lifecycle	6
4 Scan pipeline	7
5 GitHub and scheduled scanning	8
6 Threat intelligence	8
7 Data model and authorization	9
8 Scoring, usage and reporting	10
9 Security and operational boundaries	10
10 Build and operation	11
11 Implementation cross-reference	11
12 Next steps	12

1 System overview

The management console presents two related workflows. The scanner converts a submitted target into normalized vulnerability records and a numeric security posture. The security-agent workspace maintains a conversation, selects a role, enriches that role with standards and current vulnerability intelligence, and streams the model response back to the browser. Both workflows use Supabase as the control plane, but only authenticated activity is retained as user-owned database evidence.

The agent names represent four mode-specific system prompts and model selections. The implementation does not contain autonomous inter-agent delegation, tool execution or an agent-to-agent planning loop.

1.1 Implemented capabilities

Capability	Implemented behavior
Interactive scanning	Code, dependency manifests, a supplied URL string and LLM prompts are analyzed by Gemini with structured JSON findings.
Repository scanning	Public GitHub contents are enumerated recursively, filtered, downloaded and reviewed one file at a time.
Security agents	SENTINEL, CODEX, AEGIS and ASSIST provide streamed, mode-specific security responses.
Threat context	NVD CVEs, CISA KEV entries, OWASP references and critical CWE categories are assembled for prompts and finding enrichment.
Evidence management	Authenticated scans, vulnerabilities, security statistics, chat sessions, usage and schedules are stored under RLS.
Operations	Realtime dashboard refresh, finding status changes, report export, scheduled-scan configuration and a callable email-alert function.

1.2 Technology profile

Layer	Technology
Client	React 18, TypeScript, Vite, Tailwind CSS, Radix/shadcn and TanStack Query
Backend	Supabase Auth, PostgreSQL, Row Level Security, Realtime and Deno Edge Functions
Model access	Lovable AI gateway; Google Gemini 2.5 Flash and OpenAI GPT-5
Intelligence	NVD CVE 2.0 API, CISA KEV feed and static OWASP/CWE catalogs
Other integrations	GitHub Contents API and Resend email API

2 System architecture

Figure 1 separates the browser console, Supabase control plane, analysis services and external dependencies. Agent icons identify model-backed processing only; storage, UI and transport components remain icon-free.

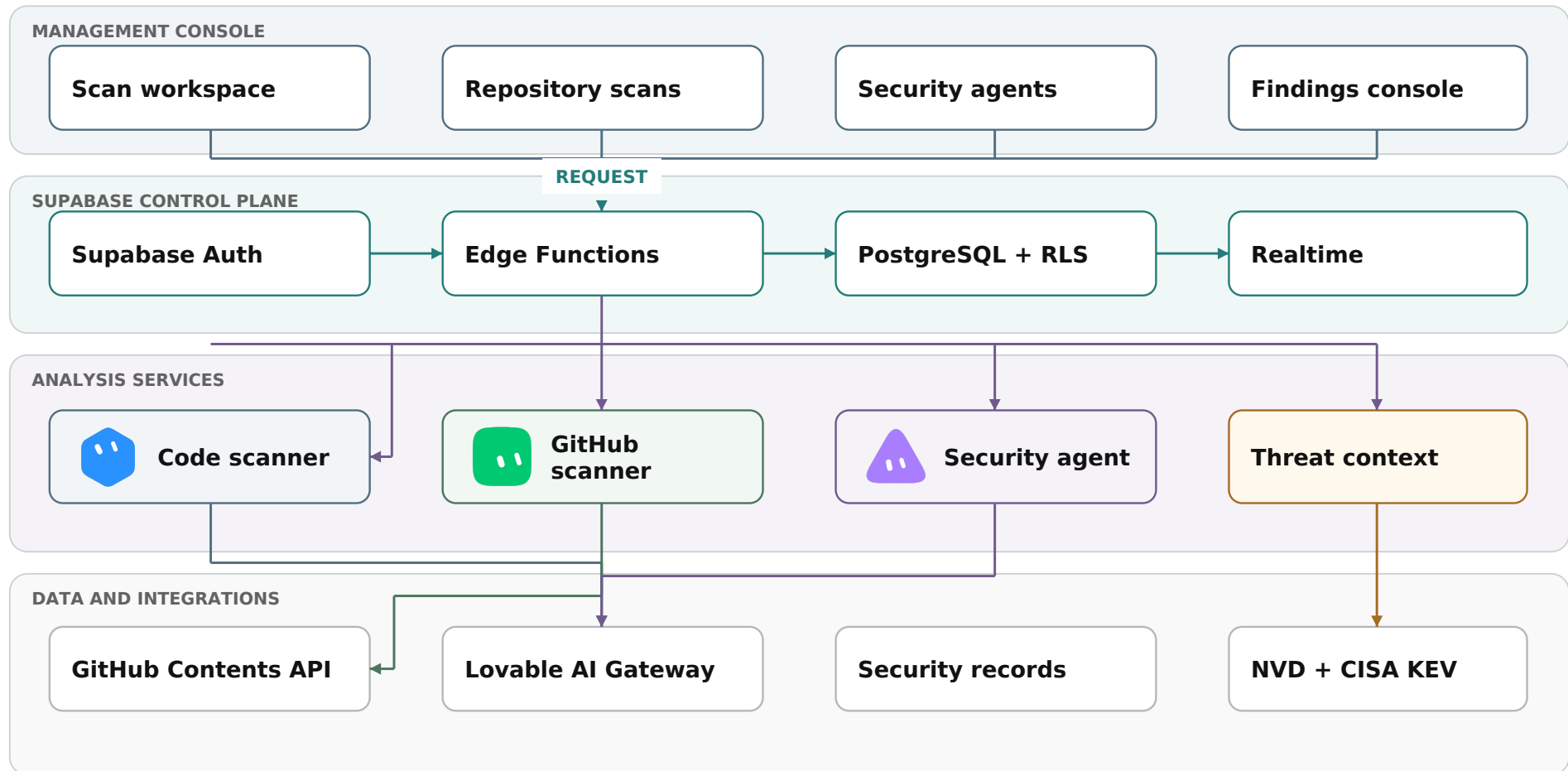


Figure 1: AEGIS.ai logical architecture and service boundaries.

2.1 Management console

The React client supplies authentication, scan input, repository scheduling, vulnerability review, metrics, report controls and the streaming agent workspace. TanStack Query and Supabase subscriptions coordinate server state, while local component state supplies a non-persistent demonstration path when a user is not authenticated.

2.2 Supabase control plane

Supabase Auth identifies users; Edge Functions mediate model, intelligence, GitHub and email integrations; PostgreSQL persists user-owned evidence; and Realtime broadcasts changes to scans, vulnerabilities and statistics. Row-level policies require the stored user identifier to match the active authenticated user.

2.3 Analysis services

The code scanner builds a scan-type-specific prompt and asks Gemini for a JSON array of findings. The GitHub scanner performs bounded repository acquisition before applying a similar per-file model review. The security-agent function fetches threat context, maps the selected mode to a system prompt and model, and relays the streamed completion. Threat-intelligence and CVE lookup functions keep live-feed retrieval outside the browser.

2.4 External boundaries

Model traffic leaves Supabase through the Lovable AI gateway. Repository content comes from the public GitHub Contents API. CVE and exploitation context is retrieved from NVD and CISA. The separate alert function uses Resend when invoked with authenticated recipient and scan details.

3 Agent request lifecycle

Figure 2 shows request preparation, enrichment, role selection, shared model-gateway execution and streamed delivery. The four agent nodes are mutually exclusive routes for one request; their outputs merge into the same SSE response path.

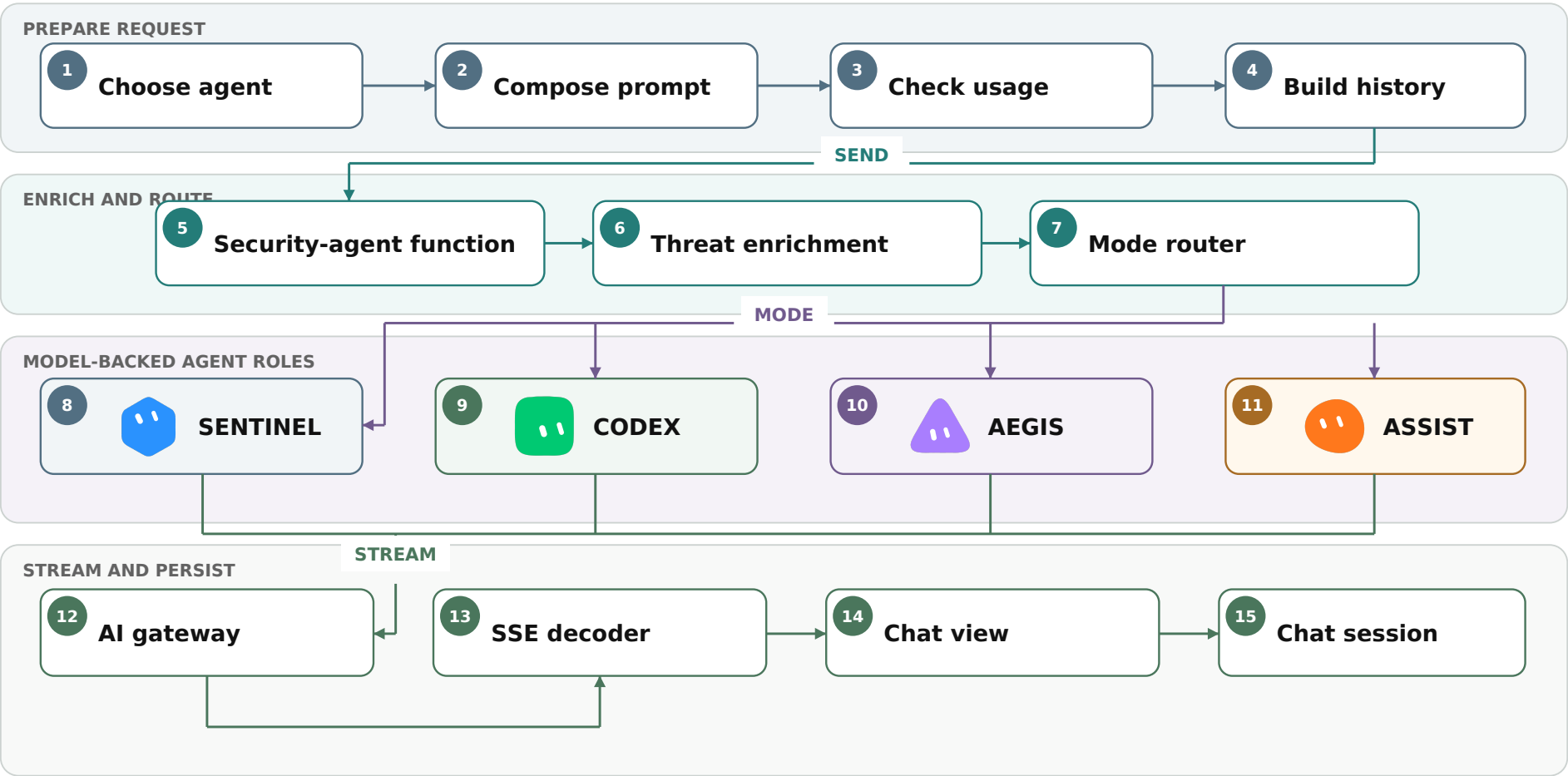


Figure 2: Mode-routed security-agent request from composition to persisted response.

3.1 Request preparation and controls

An authenticated user selects a mode and supplies a prompt. CODEX mode can read multiple text-based code files into the prompt, subject to a combined 500 KB browser-side limit. Before the request is sent, the client increments the agent usage counter. The current implementation enforces 20 accumulated requests per calendar day for the selected category, even though several internal names still refer to an hourly limit.

3.2 Threat enrichment and routing

The security-agent Edge Function requests a full threat context containing recent NVD data, CISA KEV entries and OWASP/CWE references. If that call fails, a static OWASP and CWE summary is used. A mode map then selects exactly one system prompt and model.

Agent role	Mode	Model	Responsibility
SENTINEL	security	Gemini 2.5 Flash	Fast application-security triage and vulnerability guidance
CODEX	code_review	OpenAI GPT-5	Detailed code-security review with structured remediation
AEGIS	threat_intel	OpenAI GPT-5	Threat interpretation and defensive prioritization
ASSIST	general	Gemini 2.5 Flash	General security explanation and support

3.3 Streaming and persistence

The Edge Function forwards the request to the Lovable AI gateway with streaming enabled and returns the upstream event stream. The browser decodes data lines, appends token deltas to the current assistant message and saves the completed conversation. Chat sessions are scoped by both user and agent mode; users can create, select, rename through preview changes, and delete their own history.

4 Scan pipeline

The dashboard routes GitHub scans to the repository scanner and all other interactive targets to the code-scanner Edge Function. Authenticated scans receive a running record before analysis and are updated to completed or failed; unauthenticated demonstration results remain in client memory.

4.1 Scan modes

Mode	Input	Implemented analysis
Code	Pasted source or uploaded text file	Gemini review against web, code, dependency and AI-security weakness patterns
Dependency	Manifest or dependency text	Gemini dependency assessment plus bounded recent-NVD enrichment
URL	URL string	Model analysis of the supplied string and contextual risk; the function does not fetch or actively probe the target
LLM Shield	Prompt or text	Prompt-security analysis for injection, leakage, agency, consumption and related LLM risks
GitHub	Public repository URL	Recursive file acquisition followed by per-file Gemini review

4.2 Structured findings

Scanner prompts request a JSON array. Each finding can carry a name, description, severity, category, location, remediation, suggested patch text, CWE, CVE and CVSS information. Suggested patches are evidence for human review; no code is modified or deployed by the application.

4.3 CVE enrichment

For code and dependency scans, the function derives up to three NVD search terms, queries a recent publication window and compares returned weaknesses and descriptions with model findings. The match is heuristic and can also add separate NVD Alert records, so a CVE association should be validated before remediation or disclosure.

5 GitHub and scheduled scanning

5.1 Repository acquisition

The GitHub scanner parses owner, repository and optional branch information, then walks the public Contents API recursively. It excludes common dependency, VCS and build directories; accepts source, configuration, markup and manifest extensions; rejects files at or above 100,000 bytes; uses a default request limit of 100 files and a maximum of 200; and sends at most 15,000 characters from each selected file to Gemini. Files are processed sequentially.

5.2 Manual and scheduled execution

Authenticated users can store repository schedules and run a configured scan immediately. The `scheduled-github-scan` function queries active schedules with a service-role client, invokes the GitHub scanner with a 50-file request limit, updates last-run metadata and creates user-owned scan and vulnerability records.

The repository enables `pg_cron` and `pg_net` and supplies a scheduled-scan runner, but its migrations do not register a `cron.schedule` invocation. The interface label ‘Daily at 9:00 AM UTC’ therefore describes the intended cadence; an invocation schedule still needs to be provisioned.

5.3 Repository scan limits

Control	Value
Maximum enumerated files	200
Scheduled request limit	50 files
Individual file cutoff	100,000 bytes
Model content per file	15,000 characters
Ignored paths	<code>node_modules</code> , <code>.git</code> , <code>dist</code> , <code>build</code> , <code>vendor</code> , <code>__pycache__</code> , <code>.next</code> , <code>.venv</code>

6 Threat intelligence

The threat-intelligence function maintains static OWASP Top 10 (2021), OWASP Top 10 for LLM Applications (2025) and critical CWE catalogs. It also retrieves newly published CVEs from the prior seven days, critical CVEs from the prior 30 days and a bounded set of recent CISA KEV entries. The resulting data can be returned by category or rendered as compact prompt context.

6.1 Data use

Consumer	Use
Security agents	Ground system prompts with current CVE/KEV context and stable OWASP/CWE references.
Code scanner	Supply CISA exploitation context and enrich code or dependency results with recent NVD records.
CVE lookup	Expose focused NVD retrieval through a separate Edge Function.
Dashboard findings	Display CVE identifiers, CVSS scores, severity and remediation evidence when present.

6.2 Interpretation boundary

Live-feed availability is best-effort. The agent route falls back to static context when enrichment fails, while scanner-side requests log feed failures and continue where possible. Model-generated conclusions and heuristic CVE matches remain advisory; they are not a substitute for package-version confirmation, exploitability analysis or human review.

7 Data model and authorization

Table	Purpose
security_scans	Scan type, target, status, timing and metadata
vulnerabilities	Finding detail, severity, status, evidence and remediation
security_stats	Current and previous metric values, including security score
chat_sessions	Mode-scoped conversation JSON and preview text
user_usage	Per-user, per-date agent and scan counters
scheduled_scans	Repository schedule state and last result
notification_preferences	User email and vulnerability-alert preferences
profiles	User profile data linked to authentication identity

7.1 Ownership and realtime behavior

Final migration policies restrict reads and writes to rows whose `user_id` matches `auth.uid()`. The client applies the same identifier filters when loading, editing and deleting evidence. Realtime subscriptions refresh scans, vulnerabilities and statistics after database changes. Anonymous scanner output is displayed locally instead of being inserted into shared tables.

7.2 Finding workflow

A finding begins in detected status and can be moved through review states or marked resolved or false positive. Resolution metadata and notes are retained. The dashboard fetches the 50 most recent vulnerabilities and 20 most recent scans for its active view.

8 Scoring, usage and reporting

8.1 Security score

The canonical frontend and database score starts at 100 and subtracts 15 points for each unresolved critical finding, 10 for high, 5 for medium and 2 for low, with a floor of zero. The database recalculation function counts detected and analyzing records for the current authenticated user and stores the previous score alongside the new value.

8.2 Usage control

Agent and scan requests are counted independently. Authenticated use is recorded in `user_usage` through RPC functions; unauthenticated demonstration use is tracked in `LocalStorage`. The client uses a limit of 20 accumulated requests per category for the current date. These client/RPC checks are separate from direct authorization of each Edge Function invocation. Internal hourly naming and reset metadata do not implement a rolling-hour event counter.

8.3 Reports and alerts

The dashboard can transform current finding data into report and export views. A Resend-backed Edge Function builds an HTML vulnerability alert only when critical or high findings exist and records notification preferences. No repository caller currently invokes this function automatically, so it should be treated as an implemented integration endpoint rather than an active notification pipeline.

9 Security and operational boundaries

Boundary	Implemented condition
Authentication	security-agent and code-scanner require JWT verification in Supabase configuration; other functions apply their own varying checks.
Model output	Findings and patches are advisory text; no remediation is executed.
URL analysis	A supplied URL is analyzed as text; no network probing or page retrieval occurs.
GitHub scope	Repository acquisition targets public Contents API data and does not include authenticated private-repository access.
Agent behavior	One mode selects one prompt/model route; there is no autonomous tool or agent delegation.
Scheduled runner	Uses the Supabase service role internally and therefore requires a protected invocation boundary.
CORS	Edge Functions declare permissive origins; production should restrict origins and invocation paths.

9.1 Required secrets and configuration

Variable	Purpose
VITE_SUPABASE_URL	Browser Supabase endpoint
VITE_SUPABASE_PUBLISHABLE_KEY	Browser and function invocation key
SUPABASE_URL / SUPABASE_SERVICE_ROLE_KEY	Server-side database and scheduled-runner access
LOVABLE_API_KEY	Gemini and GPT-5 gateway requests
NVD_API_KEY	Optional authenticated NVD rate allowance
RESEND_API_KEY	Email delivery in the alert function

10 Build and operation

```
npm install

# Start the Vite development server
npm run dev

# Validate and build
npm run lint
npm run build

# Preview the production bundle
npm run preview
```

10.1 Supabase deployment

Apply the repository migrations to a Supabase project, configure function secrets, deploy the Edge Functions and provide the Vite client with the target project URL and publishable key. Production operation also requires an explicit protected schedule for scheduled-github-scan if recurring scans are desired.

10.2 Functional verification

- Authenticate, create a scan and confirm the scan and finding rows are visible only to that user.
- Run each interactive scan type and verify structured error handling for invalid or incomplete input.
- Select every agent mode and confirm the expected model route, streamed response and mode-scoped session history.
- Scan a bounded public repository and verify file filters, truncation and persisted finding locations.
- Resolve or dismiss a finding and confirm score recalculation and realtime dashboard refresh.
- Invoke the scheduled runner through a protected path and verify last-run and evidence records.

11 Implementation cross-reference

Source reference: [Aegis.ai](#), revision [63b45b5e17f5](#). The review covers source and configuration; it does not certify a live deployment or every runtime outcome.

The following map connects documented behavior to the principal repository sources reviewed at commit [63b45b5e17f52047f6a35b5bb1073b3a21262184](#).

Capability	Primary implementation
Dashboard, scan routing and report UI	src/components/ThreatDashboard.tsx
Agent selection, SSE parsing and sessions	src/components/SecurityAgent.tsx
Scan, vulnerability and score state	src/hooks/useSecurityData.ts
Usage counters	src/hooks/useUserUsage.ts; 20260125210253 migration
Scheduled-scan interface	src/components/ScheduledScans.tsx
Agent prompt/model router	supabase/functions/security-agent/index.ts
Interactive scan pipeline	supabase/functions/code-scanner/index.ts
Repository acquisition and analysis	supabase/functions/github-scanner/index.ts
Threat catalogs and live feeds	supabase/functions/threat-intelligence/index.ts
Focused CVE lookup	supabase/functions/nvd-cve-lookup/index.ts
Scheduled repository runner	supabase/functions/scheduled-github-scan/index.ts
Email alert endpoint	supabase/functions/send-vulnerability-alert/index.ts
Tables, RLS, triggers and RPCs	supabase/migrations/*.sql
Function JWT configuration	supabase/config.toml
Frontend dependencies and commands	package.json

12 Next steps

Priority	Improvement	Completion criterion
1	Protect non-JWT Edge Functions	Scheduled, GitHub, intelligence and alert routes have explicit service-to-service or user authorization and restricted CORS.
2	Provision the recurring schedule	A reviewed cron or external scheduler invokes the runner at the documented cadence with monitored credentials.
3	Integrate alert delivery	Completed scans call the alert endpoint according to stored user preferences and record delivery status.
4	Align score implementations	Frontend, trigger and Edge Function paths use one shared severity-weight definition, including low findings.
5	Correct usage-window semantics	Names, counters, reset time and UI all implement either a real rolling-hour window or an explicit daily quota.
6	Add schema validation	All model JSON is validated against a versioned finding schema before persistence or rendering.
7	Add automated tests	Unit and integration suites cover routing, SSE parsing, file limits, RLS, scoring, schedule execution and feed failures.
8	Harden evidence quality	CVE matches record match rationale, package/version evidence and analyst confirmation state.